

DevOps Complete Guide

Basic to Advanced: concepts, commands and code snippets

Linux | Git | Cloud (AWS, Azure, GCP) | Docker | Kubernetes | Terraform | Ansible | Jenkins | CI/CD | Helm | Monitoring | GitOps | DevSecOps

Each chapter goes from first principles to advanced usage, with comparison tables, ready-to-run snippets, common mistakes and interview-style questions.

Contents

1. Roadmap and DevOps Foundations	5
1.1 What is DevOps?	5
1.2 Suggested learning order	5
1.3 Core DevOps principles	5
2. Linux Essentials to Advanced	7
2.1 Why Linux for DevOps	7
2.2 Navigation and file management	7
2.3 Text processing power tools	8
2.4 Users, groups and permissions	9
2.5 Processes and system monitoring	9
2.6 Services with systemd	10
2.7 Package management	10
2.8 Networking commands	10
2.9 SSH and secure remote access	11
2.10 Disks, mounts and archives	11
2.11 Shell scripting	12
2.12 Troubleshooting playbook	13
3. Git and Version Control	14
3.1 Concepts	14
3.2 Daily workflow commands	14
3.3 Branching and merging	14
3.4 Undoing things safely	15
3.5 Remotes, tags and collaboration	15
3.6 Branching strategies	16
3.7 Advanced Git	16
4. Cloud Computing Fundamentals	17
4.1 What is cloud computing?	17
4.2 Global infrastructure	17
4.3 Core services mapped across providers	18
4.4 Compute in depth	18
4.5 Networking: VPC	19
4.6 Storage	19
4.7 Identity and Access Management (IAM)	19
4.8 Databases and load balancing	20
4.9 Well-Architected pillars	20
4.10 AWS CLI cheat sheet	20

4.11 Cost management tips	20
5. Docker and Containers	21
5.1 Concepts	21
5.2 Essential commands	21
5.3 Dockerfile	22
5.4 Networking	23
5.5 Storage	23
5.6 Docker Compose	24
5.7 Security and troubleshooting	24
6. Kubernetes	26
6.1 Why Kubernetes?	26
6.2 Architecture	26
6.3 Core objects	26
6.4 kubectl essentials	27
6.5 Workload manifests	27
6.6 Configuration, secrets and storage	28
6.7 StatefulSet, Jobs and DaemonSet snippets	29
6.8 Scheduling controls	29
6.9 Autoscaling	29
6.10 Security: RBAC and policies	29
6.11 Helm: the package manager	30
6.12 Deployment strategies	30
6.13 Networking and DNS	30
6.14 Troubleshooting	30
7. Terraform: Infrastructure as Code	32
7.1 Concepts	32
7.2 HCL syntax and a first project	32
7.3 Language features	33
7.4 Modules	34
7.5 State management in depth	35
7.6 Terraform in CI/CD and best practices	35
7.7 Terraform vs other IaC tools	35
8. Ansible: Configuration Management	37
8.1 Concepts	37
8.2 Setup and ad-hoc commands	37
8.3 Playbooks	38
8.4 Control flow, loops and variables	39
8.5 Roles	39

8.6 Secrets with Ansible Vault and dynamic inventory	40
8.7 Example: install Docker and run a container	40
8.8 Performance and best practices	40
9. Jenkins and CI/CD	42
9.1 CI/CD concepts	42
9.2 Jenkins architecture	42
9.3 Declarative Pipeline (Jenkinsfile)	43
9.4 Triggers and integrations	45
9.5 Scaling, security and operations	45
9.6 Jenkins vs other CI/CD tools	45
10. Related Topics	47
10.1 GitOps and ArgoCD	47
10.2 Monitoring, logging and observability	47
10.3 DevSecOps (security in the pipeline)	47
10.4 Artifact repositories and registries	48
10.5 Networking and web server basics for DevOps	48
10.6 YAML and JSON essentials	48
10.7 Service mesh and advanced Kubernetes ecosystem (awareness)	49
10.8 Release engineering and reliability practices	49
11. End-to-End Project: From Commit to Production	50
11.1 Architecture	50
11.2 Repository layout	50
11.3 Command sequence to practice	50
12. Quick Reference and Study Plan	51
12.1 One-page command cheat sheet	51
12.2 Concept comparison summary	51
12.3 A practical 8-week study plan	51
12.4 Frequently asked scenario questions	52

1. Roadmap and DevOps Foundations

1.1 What is DevOps?

DevOps is a culture plus a set of practices that unites software **development** and IT **operations** so that software is built, tested, released and operated quickly, reliably and repeatedly. The goal is short feedback loops: commit code, have it automatically built and tested, deployed to an environment, monitored, and improved.

The DevOps loop: Plan -> Code -> Build -> Test -> Release -> Deploy -> Operate -> Monitor -> (back to Plan). Every tool in this guide fits somewhere on this loop.

Stage	Purpose	Tools covered here
Code / version control	Track and collaborate on source	Git, GitHub, GitLab
Build / CI	Compile, test, package automatically	Jenkins, GitHub Actions, GitLab CI
Package	Reproducible runnable artifacts	Docker images, registries
Provision	Create servers, networks, clusters	Terraform, cloud consoles and CLIs
Configure	Install and configure software on servers	Ansible
Orchestrate	Run containers at scale	Kubernetes, Helm
Deploy / CD	Release automatically and safely	Jenkins, ArgoCD, rolling or canary releases
Monitor	Metrics, logs, alerts	Prometheus, Grafana, ELK, CloudWatch

1.2 Suggested learning order

- 1 **Linux and shell**: every other tool runs on Linux or is operated through a terminal.
- 2 **Git**: all code, scripts and infrastructure definitions live in Git.
- 3 **Cloud basics**: servers, networks, storage, IAM, so you know what you are automating.
- 4 **Docker**: package an app once, run anywhere.
- 5 **Kubernetes**: run many containers reliably.
- 6 **Terraform**: create the cloud infrastructure from code.
- 7 **Ansible**: configure machines from code.
- 8 **Jenkins and CI/CD**: glue everything into automated pipelines.
- 9 **Monitoring, GitOps, security**: operate and harden.

1.3 Core DevOps principles

- **Automation**: anything done twice should become a script or a pipeline.
- **Infrastructure as Code (IaC)**: servers and networks are described in files, versioned in Git, reviewed like application code.
- **Immutable infrastructure**: do not patch running servers; replace them with new, tested images.
- **Continuous Integration**: merge small changes often; each merge builds and tests automatically.
- **Continuous Delivery vs Deployment**: Delivery means every change is **ready** to release (manual approval gate); Deployment means every passing change is released automatically.
- **Shift left**: test and secure early (unit tests, linting, image scanning in CI).
- **Observability**: you can understand system state from metrics, logs and traces.

- **Idempotency:** running a task many times yields the same result as running it once (Ansible and Terraform are built on this idea).

2. Linux Essentials to Advanced

2.1 Why Linux for DevOps

Most servers, containers and cloud VMs run Linux. Docker containers share the Linux kernel; Kubernetes nodes are Linux; Jenkins agents, Ansible targets and Terraform-created VMs are usually Linux. Fluency with the shell is the single most valuable DevOps skill.

Key concepts

- **Kernel:** core of the OS, manages CPU, memory, devices, processes. **Shell** (bash, zsh, sh) is the command interpreter.
- **Everything is a file:** devices (`/dev/sda`), processes (`/proc/1234`), and sockets are exposed as files.
- **Distributions:** Debian/Ubuntu (apt, `.deb`) vs RHEL/CentOS/Rocky/Amazon Linux/Fedora (dnf/yum, `.rpm`) vs Alpine (apk, tiny, popular for containers).

Filesystem hierarchy

Path	Purpose
<code>/</code>	Root of the whole tree
<code>/home/user</code>	Personal files of each user; <code>~</code> is shorthand
<code>/root</code>	Home of the root (admin) user
<code>/etc</code>	System-wide configuration files (nginx, ssh, fstab, hosts)
<code>/var</code>	Variable data: logs (<code>/var/log</code>), spool, caches, databases
<code>/usr</code>	Installed programs and libraries (<code>/usr/bin</code> , <code>/usr/local</code>)
<code>/bin</code> , <code>/sbin</code>	Essential binaries (often symlinks into <code>/usr</code>)
<code>/opt</code>	Optional third-party software (for example Jenkins agents, custom apps)
<code>/tmp</code>	Temporary files, cleared on reboot
<code>/dev</code> , <code>/proc</code> , <code>/sys</code>	Devices and virtual kernel/process information
<code>/mnt</code> , <code>/media</code>	Mount points for extra disks

2.2 Navigation and file management

```

pwd                # print working directory
ls -lah            # long list, all files, human readable sizes
cd /var/log        # change directory; cd - (previous), cd ~ (home)
mkdir -p app/{src,logs,config} # create nested dirs, brace expansion
touch file.txt     # create empty file / update timestamp
cp -r src/ backup/ # copy recursively; cp -a keeps permissions/timestamps
mv old.txt new.txt # move or rename
rm file.txt        # delete file; rm -rf dir/ (DANGEROUS, no undo)
ln -s /opt/app/current /srv/app # symbolic link; ln (no -s) = hard link
tree -L 2          # directory tree two levels deep (install tree first)
file image.bin     # detect file type
stat file.txt      # size, inode, permissions, timestamps
du -sh *           # size of each item here; du -h --max-depth=1 /var | sort -h
df -h              # filesystem usage per mount

```

Viewing and searching files

```
cat file | less           # page through; less +F file follows like tail -f
head -n 20 file; tail -n 50 file; tail -f /var/log/syslog # follow live log
wc -l file                # count lines
find / -name "*.conf" -type f 2>/dev/null                # find by name
find /var/log -type f -mtime +7 -delete                  # delete files older than 7 days
find . -type f -size +100M                                # files larger than 100 MB
grep -rn "ERROR" /var/log/app/                           # recursive search, line numbers
grep -iE "warn|fail" app.log                             # case-insensitive, regex alternation
grep -v "^#" config.conf                                # invert: hide comment lines
which python3; type ls; whereis nginx                     # locate commands
locate nginx.conf    # fast index based search (updatedb first)
```

2.3 Text processing power tools

These are used constantly for log analysis and quick automation.

```
# cut: pick columns; sort + uniq: count duplicates
cut -d: -f1,3 /etc/passwd # user and uid
awk -F: '{print $1, $3}' /etc/passwd # same with awk
awk '$9 >= 500 {c++} END {print c}' access.log # count HTTP 5xx style lines (field 9)
awk '{sum+=$1} END {print sum/NR}' numbers.txt # average
sed 's/foo/bar/g' file # substitute (stdout)
sed -i 's/8080/9090/' app.conf # edit file in place
sed -n '10,20p' file # print lines 10 to 20
sort file | uniq -c | sort -nr | head # top repeated lines
cut -d' ' -f1 access.log | sort | uniq -c | sort -nr | head -5 # top 5 client IPs
tr 'a-z' 'A-Z' < in.txt # translate characters
xargs # build commands from stdin
find . -name "*.log" | xargs gzip # compress all logs
tee out.txt # write to file AND stdout
echo "line" | sudo tee -a /etc/hosts # append to a root-owned file
jq '.items[].metadata.name' pods.json # JSON processing (very useful with cloud CLIs)
```

Pipes and redirection

Syntax	Meaning		
<code>`cmd1 \`</code>	<code>cmd2`</code>	Send stdout of cmd1 as stdin of cmd2	
<code>cmd > file / cmd >> file</code>	Overwrite / append stdout to file		
<code>cmd 2> err.txt</code>	Redirect stderr		
<code>cmd > all.txt 2>&1</code>	Redirect both stdout and stderr to the same file		
<code>cmd < file</code>	Feed file as stdin		
<code>cmd1 && cmd2</code>	Run cmd2 only if cmd1 succeeded		
<code>`cmd1 \`</code>	<code>\`</code>	<code>cmd2`</code>	Run cmd2 only if cmd1 failed
<code>cmd &</code>	Run in background		
<code>\$(cmd)</code>	Command substitution (capture output)		

2.4 Users, groups and permissions

Every file has an **owner**, a **group** and three permission sets (user, group, others), each with read (4), write (2), execute (1).

```
ls -l script.sh          # -rwxr-xr-- 1 alice dev 120 Jan 1 script.sh
chmod 755 script.sh      # rwx r-x r-x (owner full, others read+execute)
chmod u+x,g-w file       # symbolic form
chmod -R 750 /opt/app    # recursive
chown alice:dev file     # change owner and group; chown -R for directories
umask                    # default permission mask (022 -> new files 644, dirs 755)

# user management
sudo useradd -m -s /bin/bash -G sudo deploy # create user with home and shell
sudo passwd deploy
sudo usermod -aG docker deploy              # add to docker group (-a is essential!)
sudo userdel -r olduser
id deploy; groups deploy; whoami; w; last    # who is who / who is logged in
sudo -l                                     # what may I run with sudo
sudo visudo                                  # safely edit /etc/sudoers
```

Special bits: **setuid** (4xxx) runs a file as its owner; **setgid** (2xxx) inherits group on directories; **sticky bit** (1xxx, e.g. /tmp) lets only the file owner delete files. Example: **chmod 1777 /shared**. Use **getfacl/setfacl** for fine-grained ACLs.

2.5 Processes and system monitoring

A **process** is a running program with a PID. Linux uses signals to control processes.

```
ps aux | grep nginx      # snapshot of processes
ps -ef --forest          # parent-child tree
top; htop                # live view (htop is friendlier); press M=memory sort, P=cpu
pgrep -a java            # find PIDs by name
kill 1234                # SIGTERM (15): polite shutdown
kill -9 1234             # SIGKILL: force, no cleanup (last resort)
kill -HUP 1234           # SIGHUP: many daemons reload config
pkill -f "python app.py" # kill by pattern; killall nginx
nohup ./job.sh > job.log 2>&1 & # keep running after logout
jobs; fg %1; bg %1       # job control in current shell
nice -n 10 ./heavy.sh    # start with lower priority; renice -n 5 -p 1234

free -h                 # memory and swap
uptime                  # load averages (1, 5, 15 min)
vmstat 2 5              # cpu, mem, io every 2s, 5 times
iostat -xz 2            # disk IO per device (sysstat package)
lsof -i :8080           # which process uses port 8080
lsof -p 1234            # files opened by a process
dmesg -T | tail         # kernel messages (OOM kills, disk errors)
```

Load average is the average number of runnable (and uninterruptible) tasks. A load of 4 on a 4-core machine means fully busy; sustained load well above core count means saturation. **nproc** shows the core count.

Zombie and orphan processes

A **zombie** has finished but its parent has not read its exit status (shows **Z** in ps); fix by fixing/restarting the parent. An **orphan** has lost its parent and is adopted by PID 1 (init/systemd).

2.6 Services with systemd

systemd starts, stops and supervises services on modern distributions.

```
sudo systemctl status nginx          # state, recent logs
sudo systemctl start|stop|restart|reload nginx
sudo systemctl enable --now nginx    # start at boot AND now
sudo systemctl disable nginx
systemctl list-units --type=service --state=failed
systemctl daemon-reload              # after editing unit files
journalctl -u nginx -f               # follow logs of a service
journalctl -u nginx --since "1 hour ago" -p err
journalctl -b -1                     # logs from previous boot
```

A minimal custom service unit `/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Node App
After=network.target

[Service]
User=deploy
WorkingDirectory=/opt/myapp
ExecStart=/usr/bin/node server.js
Restart=on-failure
RestartSec=5
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Then: `sudo systemctl daemon-reload && sudo systemctl enable --now myapp`.

2.7 Package management

Task	Debian / Ubuntu	RHEL / Amazon Linux / Fedora	Alpine
Refresh index	<code>apt update</code>	<code>dnf makecache</code>	<code>apk update</code>
Install	<code>apt install -y nginx</code>	<code>dnf install -y nginx</code>	<code>apk add nginx</code>
Upgrade all	<code>apt upgrade -y</code>	<code>dnf upgrade -y</code>	<code>apk upgrade</code>
Remove	<code>apt remove nginx</code>	<code>dnf remove nginx</code>	<code>apk del nginx</code>
Search	<code>apt search word</code>	<code>dnf search word</code>	<code>apk search word</code>
Which package owns file	<code>dpkg -S /path</code>	<code>rpm -qf /path</code>	<code>apk info -W /path</code>
List installed	<code>dpkg -l</code>	<code>rpm -qa</code>	<code>apk info</code>

2.8 Networking commands

```
ip addr show; ip -br a              # interfaces and IPs (replaces ifconfig)
ip route; ip route get 8.8.8.8
ping -c 4 example.com               # reachability
traceroute example.com; mtr example.com # path and latency per hop
ss -tulpn                           # listening TCP/UDP sockets with process (replaces netstat)
ss -s                               # socket summary
curl -I https://example.com          # headers only
curl -sS -X POST -H "Content-Type: application/json" -d '{"a":1}' http://localhost:8080/api
curl -o file.zip -L https://host/file.zip # download, follow redirects
wget -c https://host/file.iso         # resumable download
dig example.com +short; dig MX example.com # DNS lookup; nslookup also works
nc -zv host 5432                      # test whether a TCP port is open
sudo tcpdump -i eth0 port 80 -n -c 20   # capture packets
cat /etc/resolv.conf /etc/hosts         # DNS resolver and static host entries
hostname -I                             # IP addresses of this host
```

Firewall basics

```
# Ubuntu: ufw (simple front end)
sudo ufw allow 22/tcp; sudo ufw allow 80,443/tcp; sudo ufw enable; sudo ufw status verbose
# RHEL family: firewallld
sudo firewall-cmd --permanent --add-service=http && sudo firewall-cmd --reload
# Underlying: iptables / nftables
sudo iptables -L -n -v
```

2.9 SSH and secure remote access

SSH uses public/private key pairs. The **private key never leaves your machine**; the **public key** goes into `~/.ssh/authorized_keys` on the server.

```
ssh-keygen -t ed25519 -C "me@company.com"      # generate key pair (~/.ssh/id_ed25519)
ssh-copy-id user@server                        # install public key on server
ssh user@server                               # connect
ssh -i ~/.ssh/key.pem ec2-user@1.2.3.4        # with specific key (AWS style)
scp file.txt user@server:/tmp/                # copy file; scp -r dir/ ...
rsync -avz --delete ./site/ user@server:/var/www/site/ # efficient sync
ssh -L 5432:db.internal:5432 user@bastion     # local port forward (tunnel to private DB)
ssh -J bastion user@private-host              # jump host (ProxyJump)
chmod 600 ~/.ssh/id_ed25519; chmod 700 ~/.ssh  # required permissions or ssh refuses
```

`~/.ssh/config` makes this easy:

```
Host prod-web
  HostName 10.0.1.25
  User ubuntu
  IdentityFile ~/.ssh/prod.pem
  ProxyJump bastion
```

Hardening (`/etc/ssh/sshd_config`): `PermitRootLogin no`, `PasswordAuthentication no`, `AllowUsers deploy`, then `sudo systemctl reload sshd`. Add fail2ban to block brute force attempts.

2.10 Disks, mounts and archives

```
lsblk -f                                     # block devices and filesystems
sudo fdisk -l                               # partition tables
sudo mkfs.ext4 /dev/xvdf                     # create filesystem (DESTROYS data)
sudo mkdir /data && sudo mount /dev/xvdf /data
blkid                                        # get UUID, then add to /etc/fstab for persistent mounts:
# UUID=xxxx-xxxx /data ext4 defaults,nofail 0 2
sudo mount -a                                # test fstab entries before reboot!
# archives
tar -czvf backup.tar.gz /opt/app             # create gzip tar
tar -xzf backup.tar.gz -C /restore           # extract to directory
zip -r site.zip site/; unzip site.zip
```

LVM (advanced) lets you resize volumes: physical volume (`pvcreate`) -> volume group (`vgcreate`) -> logical volume (`lvcreate`) -> `lvextend -r -L +10G /dev/vg/lv`.

2.11 Shell scripting

A script is a text file of commands. Start with a shebang and use strict mode.

```
#!/usr/bin/env bash
set -euo pipefail          # exit on error, undefined var, and pipe failure
IFS=$'\n\t'

APP_DIR="${1:-/opt/app}"  # first argument with default
LOG="/var/log/deploy.log"

log() { echo "$(date '+%F %T') [INFO] $" | tee -a "$LOG"; }

backup() {
    local ts; ts=$(date +%Y%m%d_%H%M%S)
    tar -czf "/backup/app_${ts}.tar.gz" "$APP_DIR"
    log "backup created app_${ts}.tar.gz"
}

if [[ ! -d "$APP_DIR" ]]; then
    echo "Directory $APP_DIR missing" >&2
    exit 1
fi

for svc in nginx myapp; do
    if systemctl is-active --quiet "$svc"; then
        log "$svc running"
    else
        log "$svc DOWN, restarting"; sudo systemctl restart "$svc"
    fi
done

backup
trap 'log "script exited with code $"' EXIT
```

Syntax cheat sheet

Feature	Example
Variables	<code>name="dev"; echo "\$name"</code> (always quote)
Arithmetic	<code>n=\$((5 + 3))</code>
Conditionals	<code>if [[-f file && \$x -gt 3]]; then ... elif ...; else ...; fi</code>
File tests	<code>-f file, -d dir, -e exists, -s non-empty, -x executable</code>
Number compare	<code>-eq -ne -lt -le -gt -ge</code>
Loop over list	<code>for f in *.log; do gzip "\$f"; done</code>
While read lines	<code>while read -r line; do echo "\$line"; done < hosts.txt</code>
Arrays	<code>arr=(a b c); echo "\${arr[1]}"; echo "\${#arr[@]}"</code>
Exit status	<code>\$?</code> last status; 0 means success
Arguments	<code>\$0</code> script, <code>\$1..\$9</code> , <code>\$@</code> all args, <code>\$#</code> count
Case	<code>case "\$env" in dev) ... ;; prod) ... ;; *) ... ;; esac</code>

Cron: scheduled jobs

```
crontab -e          # edit your jobs; crontab -l to list
# m h dom mon dow  command
0 2 * * * /opt/scripts/backup.sh >> /var/log/backup.log 2>&1 # daily 02:00
*/5 * * * * /opt/scripts/healthcheck.sh # every 5 minutes
30 1 * * 1-5 /opt/scripts/report.sh # weekdays 01:30
```

2.12 Troubleshooting playbook

Symptom	First commands to run	
Server slow	<code>uptime, top/htop, free -h, iostat -xz 2, dmesg -T</code>	
Disk full	<code>df -h, `du -h --max-depth=1 /`</code>	<code>sort -h, df -i (inodes), lsof +L1` (deleted but open files)</code>
Service will not start	<code>systemctl status X, journalctl -u X -n 100</code> , check config syntax (<code>nginx -t</code>)	
Cannot connect to port	<code>ss -tulpn</code> , firewall rules, security group, <code>nc -zv, curl -v</code>	
High memory	<code>`ps aux --sort=-%mem`</code>	<code>head, free -h, check swap use, OOM entries in dmesg`</code>
DNS issues	<code>dig host, /etc/resolv.conf, resolvectl status</code>	
Permission denied	<code>ls -l, id</code> , parent directory execute bit, SELinux (<code>getenforce, ausearch -m avc</code>)	

Common interview questions

- Difference between hard link and soft link? A hard link is another name for the same inode (cannot cross filesystems or point to directories); a symlink is a separate file holding a path.
- What does `kill -9` do vs `kill`? SIGKILL cannot be caught or cleaned up; SIGTERM allows graceful shutdown.
- What is an inode? Metadata structure for a file (owner, permissions, block locations); filenames map to inodes. You can run out of inodes even with free space.
- What is the difference between `&&`, `;` and `||`? Sequential regardless; run next only on success; run next only on failure.
- How do you find which process holds a port? `ss -tulpn | grep :8080` or `lsof -i :8080`.

3. Git and Version Control

3.1 Concepts

Git is a **distributed version control system**: every clone holds the full history. It stores snapshots of your project as **commits**, each identified by a SHA hash and pointing to its parent(s). A **branch** is just a movable pointer to a commit; **HEAD** points to the branch (or commit) you are on.

The three areas

Area	What it is	Moves with
Working directory	Your actual files on disk	edit files
Staging area (index)	Snapshot being prepared for the next commit	<code>git add</code>
Repository (.git)	Committed history	<code>git commit</code>
Remote	Shared repo (GitHub, GitLab, Bitbucket)	<code>git push</code> / <code>git fetch</code> / <code>git pull</code>

3.2 Daily workflow commands

```
git config --global user.name "Your Name"
git config --global user.email "you@company.com"
git config --global init.defaultBranch main
git config --global pull.rebase true          # linear history on pull

git init                                     # new repo here
git clone https://github.com/org/repo.git    # copy a remote repo
git status -sb                              # short status with branch
git add file.txt                            # stage one file; git add -p (stage hunks interactively)
git add .                                   # stage everything
git commit -m "feat: add login endpoint"
git commit --amend --no-edit                # add staged changes to the last commit (local only!)
git log --oneline --graph --decorate --all   # compact history graph
git diff                                     # unstaged changes; git diff --staged for staged
git show a1b2c3d                             # details of a commit
git blame file.txt                           # who changed each line
```

Write commit messages in the imperative mood and keep the subject under ~50 characters. A popular convention is **Conventional Commits**: `feat:`, `fix:`, `docs:`, `refactor:`, `test:`, `chore:`, which also allows automated changelogs and version bumps.

3.3 Branching and merging

```
git branch                                # list local branches; -a includes remotes
git switch -c feature/payment             # create and switch (older: git checkout -b)
git switch main
git merge feature/payment                 # merge branch into current branch
git branch -d feature/payment             # delete merged branch; -D to force
git push -u origin feature/payment        # publish and set upstream
git push origin --delete feature/payment
```

Merge vs rebase

	Merge	Rebase
Result	Creates a merge commit; history is preserved as it happened	Replays your commits on top of the target; linear history
Safe on shared branches	Yes	No (rewrites commits; never rebase commits others have pulled)
Conflicts	Resolve once	May resolve per replayed commit

	Merge	Rebase
Use for	Merging features into main (with PRs)	Cleaning up your *local* feature branch before a PR

```
git switch feature/x
git rebase main           # replay my commits on the latest main
git rebase -i HEAD~4      # interactive: squash, reword, reorder, drop last 4 commits
git merge --squash feature/x # combine whole branch into one commit
git merge --no-ff feature/x # always create a merge commit
```

Resolving conflicts

When two branches change the same lines, Git marks the file:

```
<<<<<< HEAD
timeout = 30
=====
timeout = 60
>>>>>> feature/x
```

Edit the file to the desired result, remove the markers, then `git add file` and `git commit` (merge) or `git rebase --continue`. Abort with `git merge --abort` or `git rebase --abort`.

3.4 Undoing things safely

Goal	Command	Notes
Discard unstaged edits in a file	<code>git restore file</code>	Cannot be recovered
Unstage a file	<code>git restore --staged file</code>	Keeps the edits
Undo last commit, keep changes staged	<code>git reset --soft HEAD~1</code>	History rewritten locally
Undo last commit, keep changes unstaged	<code>git reset --mixed HEAD~1</code>	Default mode
Undo last commit and throw changes away	<code>git reset --hard HEAD~1</code>	Dangerous
Undo a pushed commit safely	<code>git revert <sha></code>	Creates a new inverse commit; safe for shared branches
Recover "lost" commits	<code>git reflog</code> then <code>git switch -c rescue <sha></code>	Reflog keeps ~90 days of HEAD movement

```
git stash push -m "wip login" # shelve uncommitted work
git stash list; git stash pop  # restore and drop; git stash apply keeps it
git cherry-pick a1b2c3d        # copy one commit onto current branch
git clean -fd                 # remove untracked files/dirs (use -n for dry run first)
```

3.5 Remotes, tags and collaboration

```
git remote -v; git remote add upstream https://github.com/orig/repo.git
git fetch --all --prune # download refs without changing your files
git pull --rebase        # fetch + rebase local commits
git push --force-with-lease # safer force push (fails if remote has new commits)
git tag -a v1.2.0 -m "Release 1.2.0"; git push origin v1.2.0
git log v1.1.0..v1.2.0 --oneline # what changed between releases
```

Pull Request (GitHub) / Merge Request (GitLab): a request to merge a branch, with code review and automated CI checks. Branch protection rules can require passing checks and approvals before merging.

3.6 Branching strategies

Strategy	Idea	Best for
GitHub Flow	main is always deployable; short-lived feature branches + PR	Web apps, continuous deployment
Trunk-based development	Everyone commits to main (or tiny branches) daily; feature flags hide unfinished work	High-performing CI/CD teams
Git Flow	main , develop , feature/* , release/* , hotfix/*	Versioned/packaged releases, slower cadence
Environment branches	dev , staging , prod branches	Simple GitOps setups (but drifts easily)

3.7 Advanced Git

```
git bisect start; git bisect bad; git bisect good v1.0 # binary search for a bug-introducing commit
git worktree add ../hotfix main # second working dir for same repo
git submodule add https://github.com/org/lib.git libs/lib; git submodule update --init --recursive
git log -S"password" --oneline # find commits that added/removed a string
git shortlog -sn # commits per author
git config --global alias.lg "log --oneline --graph --decorate --all"
```

.gitignore and secrets

.gitignore lists files Git must not track (build output, **node_modules/**, **.env**, ***.tfstate**, ***.pem**). If a secret is committed, **rotating the secret is mandatory**; removing it from history (**git filter-repo** or BFG) is secondary. Use pre-commit hooks or tools like **gitleaks** to block secrets.

Git hooks

Scripts in **.git/hooks/** run on events. Example **pre-commit** that runs tests:

```
#!/bin/sh
npm test || { echo "Tests failed, commit aborted"; exit 1; }
```

Server-side events are often handled with **webhooks**: GitHub notifies Jenkins on every push so it can start a build.

Interview questions

- **git fetch** vs **git pull**? Fetch downloads only; pull = fetch + merge (or rebase).
- **reset** vs **revert**? Reset moves the branch pointer (rewrites history); revert adds a new commit that undoes a previous one.
- What is detached HEAD? HEAD points at a commit rather than a branch; new commits are orphaned unless you create a branch.
- How do you remove a file from Git but keep it locally? **git rm --cached file** then commit and add to **.gitignore**.

4. Cloud Computing Fundamentals

4.1 What is cloud computing?

Cloud computing is on-demand access to computing resources (servers, storage, databases, networking, software) over the internet with **pay-as-you-go** pricing. Instead of buying hardware, you rent exactly what you need and scale it up and down in minutes.

Five essential characteristics

On-demand self-service, broad network access, resource pooling (multi-tenancy), rapid elasticity, and measured service (metered billing).

Service models

Model	You manage	Provider manages	Examples
IaaS (Infrastructure)	OS, runtime, app, data	Hardware, virtualization, network	EC2, Azure VM, Compute Engine
PaaS (Platform)	App and data	OS, runtime, scaling	Elastic Beanstalk, App Service, App Engine
CaaS (Containers)	Containers and config	Orchestrator control plane	EKS, AKS, GKE, ECS
FaaS / Serverless	Function code only	Everything else, per-invocation billing	Lambda, Azure Functions, Cloud Functions
SaaS (Software)	Just use it	Entire stack	Gmail, Office 365, Salesforce

Deployment models

Public (shared provider infrastructure), **Private** (dedicated to one organization, e.g. OpenStack/VMware), **Hybrid** (on-prem + public connected by VPN/Direct Connect), **Multi-cloud** (more than one public provider to avoid lock-in or use best services).

Shared responsibility model

The provider secures the cloud **itself** (data centers, hardware, hypervisor); you secure what you put **in** the cloud (data, IAM, OS patches on IaaS, network rules, encryption choices).

4.2 Global infrastructure

- **Region**: a geographic area with multiple data centers (e.g. **ap-south-1** Mumbai).
- **Availability Zone (AZ)**: one or more isolated data centers inside a region. Spread workloads across 2+ AZs for high availability.
- **Edge locations / CDN**: caches content close to users (CloudFront, Azure Front Door, Cloud CDN).
- Choose a region by latency, data residency law, service availability and price.

4.3 Core services mapped across providers

Category	AWS	Azure	GCP
Virtual machines	EC2	Virtual Machines	Compute Engine
Object storage	S3	Blob Storage	Cloud Storage
Block storage	EBS	Managed Disks	Persistent Disk
File storage	EFS	Azure Files	Filestore
Virtual network	VPC	VNet	VPC
Load balancer	ELB (ALB/NLB)	Load Balancer / App Gateway	Cloud Load Balancing
Identity	IAM	Entra ID (Azure AD) + RBAC	Cloud IAM
Managed Kubernetes	EKS	AKS	GKE
Container service	ECS / Fargate	Container Apps / ACI	Cloud Run
Serverless functions	Lambda	Functions	Cloud Functions
Relational DB	RDS / Aurora	Azure SQL / Database for PostgreSQL	Cloud SQL / AlloyDB
NoSQL	DynamoDB	Cosmos DB	Firestore / Bigtable
DNS	Route 53	Azure DNS	Cloud DNS
Monitoring	CloudWatch	Azure Monitor	Cloud Monitoring
IaC native	CloudFormation	ARM / Bicep	Deployment Manager
Secrets	Secrets Manager / SSM Parameter Store	Key Vault	Secret Manager
CI/CD native	CodePipeline / CodeBuild	Azure DevOps	Cloud Build

4.4 Compute in depth

Virtual machines (EC2 example)

An instance is created from an **image** (AMI) with an **instance type** (vCPU + RAM + network), a **key pair** for SSH, a **security group**, and storage volumes.

Instance family (AWS)	Optimized for	Example
General purpose	Balanced	t3, m6i
Compute optimized	CPU-bound	c6i
Memory optimized	In-memory DBs	r6i
Storage optimized	High IOPS	i3
Accelerated	GPU / ML	p4, g5

Pricing models: On-Demand (pay per second), Reserved/Savings Plans (1 to 3 year commitment, up to ~70% off), Spot (spare capacity, up to ~90% off but can be interrupted; great for CI agents and batch jobs).

Auto Scaling: a group of identical instances behind a load balancer that grows or shrinks based on metrics (CPU, request count) or schedules, and replaces unhealthy instances automatically. **Scale up/vertical** = bigger machine; **scale out/horizontal** = more machines (preferred in cloud).

Serverless

You upload a function; the platform runs it on events (HTTP, queue message, file upload) and bills per request and duration. Pros: no server management, automatic scaling to zero. Cons: cold starts, execution time limits, harder local debugging, vendor lock-in.

4.5 Networking: VPC

A **VPC** is your private, isolated network in the cloud.

- **CIDR block:** IP range, e.g. **10.0.0.0/16** (65,536 addresses). **/24** = 256 addresses.
- **Subnet:** slice of the VPC in one AZ. **Public subnet** has a route to an **Internet Gateway (IGW)**; **private subnet** has no direct route, outbound internet goes via a **NAT Gateway** placed in a public subnet.
- **Route table:** rules saying where traffic for a CIDR goes (IGW, NAT, peering, VPN, Transit Gateway).
- **Security Group (SG):** **stateful** virtual firewall attached to an instance/ENI; allow rules only; return traffic is automatically allowed.
- **Network ACL (NACL):** **stateless** subnet-level firewall; allow and deny rules, evaluated in order; return traffic must be explicitly allowed.
- **VPC peering / Transit Gateway / VPN / Direct Connect:** ways to connect VPCs and on-premises networks.
- **Bastion host / SSM Session Manager:** controlled way into private instances.

A standard three-tier layout: public subnets (load balancer, NAT), private app subnets (EC2/containers), private data subnets (databases), replicated across at least two AZs.

Security Group	Network ACL
Instance level	Subnet level
Stateful	Stateless
Allow rules only	Allow and deny
All rules evaluated	Rules evaluated in number order

4.6 Storage

Type	Description	Use case
Object (S3)	Flat key-value of objects with metadata, HTTP API, virtually unlimited, 11 nines durability	Backups, static sites, logs, artifacts, data lakes
Block (EBS)	Raw disk attached to one instance	OS disks, databases
File (EFS/NFS)	Shared POSIX filesystem mounted by many instances	Shared content, CMS

S3 storage classes (Standard, Infrequent Access, Glacier tiers) plus **lifecycle rules** move data to cheaper tiers automatically. Enable **versioning** and **encryption**, and block public access by default.

4.7 Identity and Access Management (IAM)

IAM answers: **who** can do **what** on **which** resource.

- **User:** a person or app with long-lived credentials (prefer SSO and roles instead).
- **Group:** collection of users sharing policies.
- **Role:** an identity assumed temporarily by a user, service or another account, with short-lived credentials. EC2 instances and Lambda use roles instead of stored keys.
- **Policy:** JSON document of permissions.

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::my-app-bucket/*"
  }]
}
```

Best practices: least privilege, MFA on all human accounts (especially root), no root usage, no access keys in code, rotate credentials, use roles for workloads, enable CloudTrail (audit log of API calls).

4.8 Databases and load balancing

- **Relational (RDS):** managed MySQL/PostgreSQL/SQL Server; automated backups, patching, **Multi-AZ** standby for failover, **read replicas** for read scaling.
- **NoSQL (DynamoDB):** key-value/document, single digit ms latency at any scale, pay per request or provisioned capacity.
- **Caching (ElastiCache/Redis):** reduce DB load and latency.
- **Load balancer types:** **ALB** (layer 7, HTTP routing by path or host, WebSockets), **NLB** (layer 4, TCP/UDP, extreme performance, static IP), **GWL** (appliances). Health checks remove failed targets from rotation.

4.9 Well-Architected pillars

Operational excellence, Security, Reliability, Performance efficiency, Cost optimization, Sustainability. Key reliability terms: **High availability** (survive component failure, multi-AZ), **Fault tolerance**, **Disaster recovery** with **RPO** (how much data loss is acceptable) and **RTO** (how long recovery may take). DR strategies from cheap/slow to expensive/fast: Backup and restore, Pilot light, Warm standby, Multi-site active/active.

4.10 AWS CLI cheat sheet

```
aws configure                                # set access key, secret, region (or use SS0: aws configure s \
so)
aws sts get-caller-identity                  # who am I?
aws ec2 describe-instances --query "Reservations[].Instances[].[InstanceId,State.Name,PublicIpAddress]" \
--output table
aws ec2 run-instances --image-id ami-0abc --instance-type t3.micro --key-name mykey \
--security-group-ids sg-123 --subnet-id subnet-456 --count 1
aws ec2 stop-instances --instance-ids i-0123456789abcdef0
aws s3 ls; aws s3 mb s3://my-unique-bucket
aws s3 cp app.zip s3://my-unique-bucket/releases/
aws s3 sync ./dist s3://my-site-bucket --delete
aws iam list-users; aws iam create-role --role-name x --assume-role-policy-document file://trust.json
aws eks update-kubeconfig --name prod-cluster --region ap-south-1 # connect kubectl to EKS
aws logs tail /aws/lambda/myfn --follow
aws ecr get-login-password | docker login --username AWS --password-stdin 123456789012.dkr.ecr.ap-south- \
1.amazonaws.com
```

Azure and GCP equivalents: `az login`, `az vm list -o table`, `az aks get-credentials`; `gcloud auth login`, `gcloud compute instances list`, `gcloud container clusters get-credentials`.

4.11 Cost management tips

Tag everything (`env`, `team`, `owner`), set budgets and alerts, right-size instances, shut down dev environments at night, use Spot for fault-tolerant work, delete unattached volumes and old snapshots, and use lifecycle policies on storage. Review bills monthly (Cost Explorer).

Interview questions

- Region vs AZ? Region = geographic area; AZ = isolated data center(s) within it.
- Security group vs NACL? Stateful instance firewall vs stateless subnet firewall.
- How does a private instance reach the internet? Through a NAT Gateway in a public subnet.
- IAM user vs role? Users have long-term credentials; roles provide temporary credentials and are assumed.
- How do you make an app highly available on AWS? Multi-AZ ASG behind an ALB, Multi-AZ database, stateless app tier, health checks, backups.

5. Docker and Containers

5.1 Concepts

A **container** is an isolated process (or group of processes) that packages an application with its libraries and dependencies but **shares the host OS kernel**. It is built from a read-only **image** and starts in milliseconds.

Linux features that make containers possible:

- **Namespaces**: isolate what a process can *see* (PID, network, mount, user, hostname).
- **cgroups**: limit what a process can *use* (CPU, memory, IO).
- **Union filesystems (overlays)**: stack read-only image layers plus one thin writable layer.

Container vs virtual machine

	Container	Virtual machine
Isolation	Process level, shared kernel	Full OS per VM via hypervisor
Size	MBs	GBs
Startup	Milliseconds to seconds	Tens of seconds to minutes
Density	Hundreds per host	Few per host
Security boundary	Weaker (kernel shared)	Stronger
Best for	Microservices, CI, scaling	Different OS kernels, strong isolation

Docker architecture

- **Docker client** (`docker` CLI) talks to the **Docker daemon** (`dockerd`) via a REST API/socket (`/var/run/docker.sock`). The daemon uses **containerd** and **runc** to run containers.
- **Image**: immutable template. **Container**: running instance of an image. **Registry**: storage for images (Docker Hub, ECR, GHCR, Harbor). **Tag**: a label like `nginx:1.27`; **digest** (`sha256:...`) pins an exact image.
- Standards: OCI (Open Container Initiative) image and runtime specs mean Podman, containerd and CRI-O are interchangeable in many cases.

5.2 Essential commands

```
docker version; docker info
docker pull nginx:1.27-alpine
docker run -d --name web -p 8080:80 nginx:1.27-alpine      # detached, map host 8080 -> container 80
docker run -it --rm ubuntu:24.04 bash                    # interactive shell, auto remove on exit
docker run -d -e DB_HOST=db -v pgdata:/var/lib/postgresql/data --memory=512m --cpus=1 postgres:16
docker ps; docker ps -a                                  # running; all including stopped
docker logs -f --tail 100 web                             # follow logs
docker exec -it web sh                                    # shell into running container
docker inspect web | jq '.[0].NetworkSettings.IPAddress'
docker stats                                              # live CPU/mem per container
docker stop web; docker start web; docker restart web; docker rm -f web
docker images; docker rmi nginx:1.27-alpine
docker cp web:/etc/nginx/nginx.conf ./nginx.conf         # copy files in/out
docker system df; docker system prune -af --volumes      # reclaim disk (careful!)
docker login; docker tag myapp:1.0 myrepo/myapp:1.0; docker push myrepo/myapp:1.0
```

`-p HOST:CONTAINER` publishes a port. `-v` mounts storage. `-e/--env-file` pass configuration. `--restart unless-stopped` makes containers survive reboots. Containers are **ephemeral**: data written inside a container is lost when it is removed unless stored in a volume.

5.3 Dockerfile

A Dockerfile is the recipe for building an image. Each instruction creates a **layer**; unchanged layers are cached.

Instruction	Purpose
FROM	Base image (start of each stage)
WORKDIR	Set working directory
COPY / ADD	Copy files in (ADD also extracts tarballs and fetches URLs; prefer COPY)
RUN	Execute a command at build time (install packages, compile)
ENV / ARG	Runtime environment variable / build-time variable
EXPOSE	Document the listening port (does not publish it)
USER	Run as non-root user
CMD	Default command (easily overridden at docker run)
ENTRYPOINT	Fixed executable; CMD supplies default arguments to it
HEALTHCHECK	Command Docker runs to test container health
VOLUME	Declare mount point for persistent data

Example: Node.js app with **multi-stage build** and best practices.

```
# ---- build stage ----
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci                # deps first: this layer is cached until package.json changes
COPY . .
RUN npm run build

# ---- runtime stage (small, no build tools) ----
FROM node:20-alpine
ENV NODE_ENV=production
WORKDIR /app
COPY --from=build /app/dist ./dist
COPY --from=build /app/package*.json ./
RUN npm ci --omit=dev && npm cache clean --force
USER node                # do not run as root
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://localhost:3000/health || exit 1
CMD ["node", "dist/server.js"]
```

Java Spring Boot example:

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src ./src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:21-jre-alpine
COPY --from=build /src/target/*.jar /app/app.jar
USER 1001
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75", "-jar", "/app/app.jar"]
```

Build and run:

```
docker build -t myapp:1.0 .
docker build -t myapp:1.0 -f Dockerfile.prod --build-arg VERSION=1.0 .
docker buildx build --platform linux/amd64,linux/arm64 -t me/myapp:1.0 --push . # multi-arch
```

.dockerignore (like **.gitignore**) keeps **node_modules**, **.git**, ***.log**, **.env** out of the build context so builds are fast and secrets do not leak.

CMD vs ENTRYPOINT, shell vs exec form

- **Exec form CMD** `["node", "app.js"]` runs the process directly as PID 1 and receives signals (graceful shutdown works). **Shell form CMD** `node app.js` wraps in `/bin/sh -c` and often swallows SIGTERM.
- **ENTRYPOINT** `["java", "-jar", "app.jar"]` + **CMD** `["--spring.profiles.active=dev"]` -> arguments can be overridden at run time while the executable stays fixed.

Image optimization and best practices

- 1 Use small, specific base images (`-alpine`, `-slim`, distroless) and **pin versions**, never rely on `latest`.
- 2 Order instructions from least to most frequently changing to maximize layer caching.
- 3 Use **multi-stage builds** to keep compilers and tests out of the final image.
- 4 Run as **non-root**; drop capabilities; use read-only root filesystem where possible.
- 5 One main process per container; log to stdout/stderr so the platform collects logs.
- 6 Combine related **RUN** steps and clean caches (`apt-get clean && rm -rf /var/lib/apt/lists/*`).
- 7 Scan images (Trivy, Gype, Docker Scout) and sign them (cosign).
- 8 Never bake secrets into images; inject at runtime.

5.4 Networking

Driver	Behavior
bridge (default)	Private network on the host; containers talk by IP; user-defined bridges add DNS by container name
host	Container shares the host network stack (no port mapping, fastest)
none	No networking
overlay	Multi-host network (Docker Swarm)
macvlan	Container gets its own MAC/IP on the physical LAN

```
docker network create appnet
docker run -d --name db --network appnet postgres:16
docker run -d --name api --network appnet -e DB_HOST=db myapi:1.0 # reach DB at hostname "db"
docker network ls; docker network inspect appnet
```

5.5 Storage

Type	Command	Use
Volume (managed by Docker)	<code>-v pgdata:/var/lib/postgresql/data</code>	Databases, persistent data; preferred
Bind mount	<code>-v \$(pwd)/src:/app/src</code>	Live code reload in development
tmpfs	<code>--tmpfs /tmp</code>	In-memory temp data, secrets

```
docker volume create pgdata; docker volume ls; docker volume inspect pgdata
docker run --rm -v pgdata:/data -v $(pwd):/backup alpine tar czf /backup/pg.tgz -C /data . # backup
```

5.6 Docker Compose

Compose defines and runs multi-container apps from one YAML file.

```
# compose.yaml
services:
  api:
    build: ./api
    ports: ["8080:8080"]
    environment:
      DB_HOST: db
      DB_PASSWORD_FILE: /run/secrets/db_pass
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped
    secrets: [db_pass]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD_FILE: /run/secrets/db_pass
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 10s
      retries: 5
  cache:
    image: redis:7-alpine
volumes:
  pgdata:
secrets:
  db_pass:
    file: ./db_pass.txt
```

```
docker compose up -d --build      # build and start in background
docker compose ps; docker compose logs -f api
docker compose exec db psql -U postgres
docker compose down -v           # stop and remove containers, networks and volumes
```

Compose creates a default network so services reach each other by service name (**db**, **cache**). Use profiles and override files (**compose.override.yaml**) for dev vs prod differences.

5.7 Security and troubleshooting

- Do not mount **/var/run/docker.sock** into containers (gives root-equivalent on the host). Do not use **--privileged** unless unavoidable.
- Limit resources (**--memory**, **--cpus**, **--pids-limit**), use **--read-only**, **--cap-drop ALL**, **--security-opt no-new-privileges**.
- **Rootless Docker/Podman** reduce the blast radius of a compromise.

Problem	Check
Container exits immediately	docker logs , docker inspect --format '{{.State.ExitCode}}' ; exit 137 = killed (often OOM), 1 = app error
Cannot reach app	Port mapping, app binding to 0.0.0.0 (not 127.0.0.1), firewall
Disk fills up	docker system df , prune unused images/volumes, log rotation (--log-opt max-size=10m)
Build slow	Layer order, .dockerignore , BuildKit cache mounts

Docker vs Podman vs containerd

Docker = full developer experience with a daemon. **Podman** is daemonless and rootless by default and CLI-compatible (**alias** `docker=podman`). **containerd/CRI-O** are lean runtimes used by Kubernetes (Kubernetes removed the Docker shim in 1.24, but Docker-built images still run perfectly).

Interview questions

- Image vs container? Image is the read-only template; container is a running instance with a writable layer.
- COPY vs ADD? COPY copies files only; ADD also extracts archives and downloads URLs.
- How do you reduce image size? Multi-stage builds, minimal base images, clean caches, `.dockerignore`.
- How does data persist? Volumes or bind mounts, external storage.
- What is a dangling image? An untagged layer left from a rebuild; remove with `docker image prune`.

6. Kubernetes

6.1 Why Kubernetes?

Running one container is easy; running hundreds across many machines needs **orchestration**: scheduling, self-healing, scaling, service discovery, rolling updates, configuration and secrets. Kubernetes (K8s) is the open-source standard for that. You declare the **desired state** in YAML; controllers continuously reconcile **actual state** toward it.

6.2 Architecture

Control plane (the brain):

- **kube-apiserver**: front door; every component and **kubectl** talks to it (REST).
- **etcd**: consistent key-value store holding all cluster state (back it up!).
- **kube-scheduler**: places pods on nodes based on resources, affinity, taints.
- **kube-controller-manager**: runs controllers (Deployment, ReplicaSet, Node, Job...) that reconcile state.
- **cloud-controller-manager**: integrates with cloud load balancers, volumes, nodes.

Worker node:

- **kubelet**: agent that makes sure the pod's containers run and report health.
- **kube-proxy**: programs networking rules (iptables/IPVS) for Services.
- **Container runtime**: containerd or CRI-O.

Managed offerings (EKS, AKS, GKE) run the control plane for you. Local learning: **minikube**, **kind**, **k3d**, Docker Desktop.

6.3 Core objects

Object	Purpose
Pod	Smallest deployable unit: one or more containers sharing network (one IP) and volumes. Ephemeral.
ReplicaSet	Keeps N identical pods running
Deployment	Declarative updates for stateless apps: manages ReplicaSets, rolling updates and rollbacks
StatefulSet	Pods with stable identity (db-0 , db-1), stable storage; for databases and queues
DaemonSet	One pod on every (or selected) node: log collectors, monitoring agents
Job / CronJob	Run-to-completion tasks / scheduled tasks
Service	Stable virtual IP and DNS name that load-balances to matching pods
Ingress	HTTP(S) routing (host/path) into the cluster, needs an Ingress controller (NGINX, Traefik)
ConfigMap / Secret	Configuration / sensitive data injected as env vars or files
PersistentVolume (PV) / PVC	Storage resource / a request for storage; StorageClass provisions dynamically
Namespace	Virtual cluster for isolation, quotas and RBAC
HPA / VPA	Horizontal / vertical autoscaling of pods
ServiceAccount, Role, RoleBinding	Identity and RBAC permissions
NetworkPolicy	Firewall rules between pods

6.4 kubectl essentials

```

kubectl config get-contexts; kubectl config use-context prod; kubectl config set-context --current --nam \
  espace=dev
kubectl get nodes -o wide
kubectl get pods -A # all namespaces
kubectl get pods -n dev -l app=web -o wide --watch
kubectl describe pod web-abc123 # events: great for troubleshooting
kubectl logs web-abc123 -c app -f --previous # --previous = logs of crashed container
kubectl exec -it web-abc123 -- sh
kubectl apply -f deploy.yaml # create or update declaratively
kubectl delete -f deploy.yaml
kubectl create deployment web --image=nginx --replicas=3 --dry-run=client -o yaml > deploy.yaml
kubectl scale deployment web --replicas=5
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web; kubectl rollout history deployment/web
kubectl rollout undo deployment/web # rollback
kubectl port-forward svc/web 8080:80 # test a service locally
kubectl top pods; kubectl top nodes # needs metrics-server
kubectl get events --sort-by=.lastTimestamp
kubectl explain deployment.spec.strategy # built-in docs for any field
kubectl auth can-i create pods --as=system:serviceaccount:dev:ci

```

6.5 Workload manifests

Deployment + Service

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: dev
  labels: { app: web }
spec:
  replicas: 3
  selector:
    matchLabels: { app: web } # must match template labels
  strategy:
    type: RollingUpdate
    rollingUpdate: { maxSurge: 1, maxUnavailable: 0 }
  template:
    metadata:
      labels: { app: web }
    spec:
      containers:
        - name: web
          image: myrepo/web:1.4.2
          ports: [{ containerPort: 8080 }]
          env:
            - name: DB_HOST
              valueFrom: { configMapKeyRef: { name: web-config, key: db_host } }
            - name: DB_PASSWORD
              valueFrom: { secretKeyRef: { name: web-secret, key: password } }
          resources:
            requests: { cpu: 100m, memory: 128Mi } # used for scheduling
            limits: { cpu: 500m, memory: 256Mi } # hard ceiling
          readinessProbe: # gets traffic only when ready
            httpGet: { path: /health/ready, port: 8080 }
            initialDelaySeconds: 5
          livenessProbe: # restarts container when unhealthy
            httpGet: { path: /health/live, port: 8080 }
            initialDelaySeconds: 15
---
apiVersion: v1
kind: Service
metadata: { name: web, namespace: dev }
spec:
  type: ClusterIP
  selector: { app: web }
  ports: [{ port: 80, targetPort: 8080 }]

```

Service types

Type	Behavior
ClusterIP (default)	Internal-only virtual IP
NodePort	Opens a port (30000-32767) on every node
LoadBalancer	Provisions a cloud load balancer (AWS ELB, Azure LB)
ExternalName	DNS CNAME alias to an external hostname
Headless (clusterIP: None)	No virtual IP; DNS returns pod IPs (used by StatefulSets)

Ingress

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations: { cert-manager.io/cluster-issuer: letsencrypt }
spec:
  ingressClassName: nginx
  tls: [{ hosts: [app.example.com], secretName: web-tls }]
  rules:
    - host: app.example.com
      http:
        paths:
          - { path: /api, pathType: Prefix, backend: { service: { name: api, port: { number: 80 } } } }
          - { path: /, pathType: Prefix, backend: { service: { name: web, port: { number: 80 } } } }
```

6.6 Configuration, secrets and storage

```

apiVersion: v1
kind: ConfigMap
metadata: { name: web-config }
data:
  db_host: db.dev.svc.cluster.local
  app.properties: |
    log.level=info
---
apiVersion: v1
kind: Secret
metadata: { name: web-secret }
type: Opaque
stringData: { password: "s3cr3t" }      # stored base64-encoded (NOT encrypted by default)
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: { name: data }
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources: { requests: { storage: 10Gi } }
```

Kubernetes Secrets are only base64 encoded. Enable encryption at rest for etcd, restrict RBAC, and prefer external secret managers (External Secrets Operator, Vault, cloud KMS/Secrets Manager).

Access modes: **RWO** (one node read-write), **ROX** (many nodes read-only), **RWX** (many nodes read-write, needs NFS/EFS-type storage).

6.7 StatefulSet, Jobs and DaemonSet snippets

```
apiVersion: batch/v1
kind: CronJob
metadata: { name: nightly-backup }
spec:
  schedule: "0 2 * * *"
  concurrencyPolicy: Forbid
  jobTemplate:
    spec:
      backoffLimit: 2
      template:
        spec:
          restartPolicy: OnFailure
          containers:
            - { name: backup, image: myrepo/backup:1.0, args: ["--target", "s3://bkp"] }
```

StatefulSet uses **serviceName**: `<headless-svc>` and **volumeClaimTemplates** so every replica gets its own PVC that survives rescheduling.

6.8 Scheduling controls

- **nodeSelector / node affinity**: run only on nodes with certain labels. **Pod anti-affinity**: spread replicas across nodes or zones.
- **Taints and tolerations**: a taint repels pods from a node (`kubectl taint nodes gpu1 gpu=true:NoSchedule`); only pods with a matching toleration may land there.
- **Topology spread constraints**: even distribution across zones.
- **PodDisruptionBudget**: guarantees a minimum number of pods stay up during voluntary disruptions like node drains.
- **PriorityClass**: important pods may preempt less important ones.

6.9 Autoscaling

Scaler	What it scales	Based on
HPA	Number of pod replicas	CPU, memory, custom metrics
VPA	Pod CPU/memory requests	Observed usage
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods

```
kubectl autoscale deployment web --cpu-percent=70 --min=3 --max=10
```

HPA needs **resource requests** defined and metrics-server installed.

6.10 Security: RBAC and policies

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: { name: pod-reader, namespace: dev }
rules:
  - apiGroups: [""]
    resources: ["pods", "pods/log"]
    verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: { name: read-pods, namespace: dev }
subjects: [{ kind: User, name: alice, apiGroup: rbac.authorization.k8s.io }]
roleRef: { kind: Role, name: pod-reader, apiGroup: rbac.authorization.k8s.io }
```

Role/RoleBinding are namespaced; **ClusterRole/ClusterRoleBinding** are cluster-wide. Apply **NetworkPolicies** (default deny, then allow), Pod Security Standards (restricted), non-root users, read-only filesystems, image scanning

and admission policies (OPA Gatekeeper, Kyverno).

6.11 Helm: the package manager

Helm packages manifests as a **chart** with templated YAML and a **values.yaml**; an installed chart is a **release**.

```
helm repo add bitnami https://charts.bitnami.com/bitnami && helm repo update
helm search repo nginx
helm install web bitnami/nginx -n dev --create-namespace -f my-values.yaml --set replicaCount=3
helm upgrade --install web ./mychart -n dev -f values-prod.yaml
helm list -A; helm history web; helm rollback web 1; helm uninstall web
helm template ./mychart | kubectl apply --dry-run=client -f - # render locally
helm create mychart # scaffold a chart
```

Template snippet: `image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"`. **Kustomize** (built into kubectl via `kubectl apply -k`) is the template-free alternative using base + overlays with patches.

6.12 Deployment strategies

Strategy	How	Trade-off
Rolling update (default)	Replace pods gradually	Zero downtime, both versions run briefly
Recreate	Stop all, then start new	Downtime; simple, avoids version mix
Blue/Green	Two full environments, switch Service selector or LB	Instant rollback, double resources
Canary	Send small % of traffic to new version	Low risk; needs ingress/service mesh (Argo Rollouts, Istio)

6.13 Networking and DNS

Every pod gets its own IP (flat network via a **CNI plugin**: Calico, Cilium, Flannel, AWS VPC CNI). Service DNS: `<service>.<namespace>.svc.cluster.local`; pods in the same namespace can simply use `<service>`. kube-proxy and CoreDNS provide load balancing and name resolution.

6.14 Troubleshooting

Status	Meaning and fix
Pending	Not scheduled: insufficient resources, unbound PVC, taints. <code>kubectl describe pod</code> Events
ImagePullBackOff / ErrImagePull	Wrong image name/tag, or missing <code>imagePullSecrets</code> for private registry
CrashLoopBackOff	App keeps crashing: <code>kubectl logs --previous</code> , wrong command/config, failing probes
OOMKilled	Exceeded memory limit: raise limit or fix leak
CreateContainerConfigError	Missing ConfigMap/Secret key
Service has no endpoints	Selector does not match pod labels; pods not Ready. <code>kubectl get endpoints svc</code>
Evicted	Node under pressure (disk/memory)

Debug flow: `get -> describe -> logs -> exec / kubectl debug -> check Events, Service endpoints, DNS (nslookup from a debug pod), NetworkPolicy`.

Kubernetes vs Docker Swarm vs ECS

	Kubernetes	Docker Swarm	AWS ECS
Complexity	High, very powerful	Low	Medium
Ecosystem	Largest (Helm, operators, service meshes)	Small	AWS-integrated
Portability	Any cloud or on-prem	Any	AWS only
Best for	Production at scale	Simple small clusters	Teams all-in on AWS

Interview questions

- Pod vs container? Pod wraps one or more containers sharing network and storage.
- Deployment vs StatefulSet? Interchangeable stateless replicas vs stable identity and storage.
- Liveness vs readiness probe? Liveness restarts unhealthy containers; readiness removes pods from Service endpoints until ready. A **startup probe** protects slow-starting apps.
- ClusterIP vs NodePort vs LoadBalancer vs Ingress? Internal; node port; cloud LB per service; one HTTP entry point routing to many services.
- What happens when you run `kubectl apply`? API server authenticates, authorizes, admits and stores in etcd; controllers create ReplicaSet and Pods; scheduler binds to nodes; kubelet starts containers.
- Requests vs limits? Requests guarantee and drive scheduling; limits cap usage (CPU is throttled, memory excess is OOMKilled).

7. Terraform: Infrastructure as Code

7.1 Concepts

Terraform (HashiCorp; the open-source fork is **OpenTofu**) lets you describe cloud and SaaS infrastructure in declarative **HCL** files. You state *what* you want; Terraform computes *how* to get there by comparing your code, the **state file**, and the real world.

Benefits: repeatable environments, peer-reviewed changes in Git, drift detection, a preview of changes before applying (**plan**), and one workflow across AWS, Azure, GCP, Kubernetes, GitHub, Datadog and 3000+ providers.

Key terms

Term	Meaning
Provider	Plugin that talks to an API (aws, azure, google, kubernetes)
Resource	A managed object (EC2 instance, S3 bucket, VPC)
Data source	Read-only lookup of existing things (latest AMI, existing VPC)
Variable / Output	Inputs to and results from a module
Module	Reusable folder of Terraform code
State	JSON mapping of your config to real resource IDs (terraform.tfstate)
Backend	Where state is stored (local, S3, Azure Blob, Terraform Cloud)
Workspace	Multiple state instances from the same configuration

Workflow

```
terraform init      # download providers/modules, configure backend
terraform fmt -recursive # format code
terraform validate  # syntax and internal consistency check
terraform plan -out=tfplan # preview: + create, ~ update, - destroy, -/+ replace
terraform apply tfplan   # execute the saved plan (or apply without file, with prompt)
terraform destroy      # tear everything down
terraform output; terraform show; terraform state list
```

Never edit **terraform.tfstate** by hand and **never commit it to Git**: it can contain secrets. Use a remote backend.

7.2 HCL syntax and a first project

Layout: **main.tf**, **variables.tf**, **outputs.tf**, **providers.tf**, **terraform.tfvars**.

```
# providers.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
  backend "s3" {
    # remote state with locking
    bucket      = "mycompany-tfstate"
    key         = "prod/network/terraform.tfstate"
    region      = "ap-south-1"
    dynamodb_table = "tf-locks" # prevents two applies at once
    encrypt     = true
  }
}
provider "aws" {
  region = var.region
  default_tags { tags = { Environment = var.env, ManagedBy = "terraform" } }
}
```

```
# variables.tf
variable "region"      { type = string, default = "ap-south-1" }
variable "env"         { type = string }
variable "instance_type" { type = string, default = "t3.micro" }
variable "allowed_cidrs" { type = list(string), default = ["0.0.0.0/0"] }
variable "db_password"  { type = string, sensitive = true } # hidden in output

# main.tf
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"]          # Canonical
  filter {
    name     = "name"
    values   = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web" {
  name      = "${var.env}-web-sg"
  vpc_id    = module.vpc.vpc_id
  ingress {
    from_port = 80, to_port = 80, protocol = "tcp", cidr_blocks = var.allowed_cidrs
  }
  egress {
    from_port = 0, to_port = 0, protocol = "-1", cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  count          = 2
  ami            = data.aws_ami.ubuntu.id
  instance_type  = var.instance_type
  subnet_id      = module.vpc.public_subnets[count.index]
  vpc_security_group_ids = [aws_security_group.web.id]
  user_data      = file("${path.module}/install_nginx.sh")
  tags           = { Name = "${var.env}-web-${count.index}" }

  lifecycle {
    create_before_destroy = true
  }
}

# outputs.tf
output "web_ips" {
  value     = aws_instance.web[*].public_ip
  description = "Public IPs of web servers"
}
```

Passing values: `terraform apply -var="env=dev", -var-file=prod.tfvars`, or environment variables `TF_VAR_env=dev`.

7.3 Language features

Types and expressions

Primitives: `string`, `number`, `bool`; collections: `list(...)`, `set(...)`, `map(...)`; structural: `object({...})`, `tuple([...])`. Reference resources as `aws_instance.web.id`, variables as `var.name`, locals as `local.name`, module outputs as `module.vpc.vpc_id`.

```

locals {
  name_prefix = "${var.env}-app"
  common_tags = { Project = "shop", Owner = "platform-team" }
}

# conditional
instance_type = var.env == "prod" ? "m5.large" : "t3.micro"

# for_each over a map: stable keys (better than count when items may be added/removed)
variable "buckets" { default = { logs = "private", assets = "public-read" } }
resource "aws_s3_bucket" "b" {
  for_each = var.buckets
  bucket   = "${local.name_prefix}-${each.key}"
}

# for expression and functions
output "upper_names" { value = [for n in var.names : upper(n) if n != ""] }
# useful functions: length, merge, lookup, join, split, format, element, flatten, toset, tomap, cidrsubn \
# et
cidr = cidrsubnet("10.0.0.0/16", 8, 1) # 10.0.1.0/24

# dynamic block: generate repeated nested blocks
dynamic "ingress" {
  for_each = var.ports
  content { from_port = ingress.value, to_port = ingress.value, protocol = "tcp", cidr_blocks = ["10.0.0 \
    .0/8"] }
}

```

count addresses resources by index (removing item 0 shifts the rest and can cause recreation); **for_each** addresses by key, which is safer.

Meta-arguments

depends_on (explicit dependency; implicit ones come from references), **count**, **for_each**, **provider** (alias for multiple regions/accounts), **lifecycle** (**create_before_destroy**, **prevent_destroy**, **ignore_changes**, **replace_triggered_by**), and **precondition/postcondition** checks.

7.4 Modules

A module is any folder with **.tf** files. The root module calls child modules to build reusable blocks.

```

module "vpc" {
  source = "terraform-aws-modules/vpc/aws" # registry module
  version = "~> 5.0"
  name    = "${var.env}-vpc"
  cidr    = "10.0.0.0/16"
  azs     = ["ap-south-1a", "ap-south-1b"]
  public_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  private_subnets = ["10.0.11.0/24", "10.0.12.0/24"]
  enable_nat_gateway = true
}

module "app" {
  source      = "../modules/app" # local module
  instance_type = "t3.small"
  subnet_ids  = module.vpc.private_subnets
}

```

Good module design: small focused inputs, sensible defaults, documented outputs, pinned versions, and no provider configuration inside child modules.

7.5 State management in depth

- **Remote backend + locking** gives a single source of truth and prevents concurrent applies. Enable bucket versioning and encryption.
- **Split state** by environment and by layer (network, data, app) to limit blast radius and speed up plans.

```
terraform state list
terraform state show aws_instance.web[0]
terraform state mv aws_instance.old aws_instance.new      # refactor without recreating
terraform state rm aws_instance.web[0]                    # stop managing (resource stays)
terraform import aws_s3_bucket.logs my-existing-bucket    # adopt an existing resource
terraform force-unlock <LOCK_ID>                          # only if a crashed run left a lock
terraform workspace new staging; terraform workspace select staging; terraform workspace list
terraform apply -replace=aws_instance.web[0]              # force recreate one resource
terraform plan -refresh-only                              # detect drift
```

Modern HCL also supports `moved { from = ..., to = ... }` and `import { to = ..., id = ... }` blocks so refactors and imports are reviewed in a plan like normal code.

Handling secrets

Mark variables `sensitive = true`, pull secrets from Secrets Manager/Vault data sources (note: values still land in state, so protect state), use short-lived credentials via OIDC in CI, and never commit `*.tfvars` containing secrets.

7.6 Terraform in CI/CD and best practices

- 1 Run `fmt -check`, `validate`, `tflint`, and security scanners (`tfsec/trivy config`, `checkov`) on each pull request.
- 2 Post `terraform plan` output in the PR; apply only from `main` after approval.
- 3 Pin provider and module versions; commit `.terraform.lock.hcl`.
- 4 Use separate accounts/state per environment (dev, staging, prod). **Terragrunt** helps keep multi-environment code DRY.
- 5 Tag everything and use consistent naming.
- 6 Policy as code: Sentinel or OPA to block risky changes (public buckets, oversized instances).

```
# GitHub Actions fragment
- uses: hashicorp/setup-terraform@v3
- run: terraform init -input=false
- run: terraform fmt -check && terraform validate
- run: terraform plan -input=false -out=tfplan
```

7.7 Terraform vs other IaC tools

	Terraform/OpenTofu	CloudFormation	Pulumi	Ansible
Language	HCL (declarative)	JSON/YAML	Python, TypeScript, Go, C#	YAML playbooks
Clouds	Multi-cloud, 3000+ providers	AWS only	Multi-cloud	Multi-cloud modules
State	Explicit state file	Managed by AWS stacks	State backend (managed or self)	Stateless
Primary purpose	Provisioning infrastructure	AWS provisioning	Provisioning in real languages	Configuration and orchestration
Strength	Ecosystem, plan/apply workflow	Deep AWS integration, rollback	Loops, tests, real code	Agentless, simple

Interview questions

- What is state and why remote? Maps config to real resources; remote gives sharing, locking, durability.
- `count` vs `for_each`? Index vs key identity; prefer `for_each` for stable addresses.
- What is drift and how do you fix it? Real infra differs from state; run `plan/refresh-only`, then apply to reconcile or update the code.
- `terraform taint` replacement? Deprecated; use `apply -replace=ADDRESS`.
- What happens if two people apply simultaneously? State locking (e.g. DynamoDB or native S3 locking) blocks the second.

8. Ansible: Configuration Management

8.1 Concepts

Ansible automates configuration, application deployment and orchestration. It is **agentless** (connects over SSH for Linux, WinRM for Windows, needs only Python on managed nodes), **declarative** and **idempotent**, and uses human-readable **YAML**. Key terms:

Term	Meaning
Control node	Machine where you run Ansible (Linux/macOS/WSL)
Managed nodes	Servers being configured
Inventory	List of hosts and groups
Module	Unit of work (apt , copy , service , user)
Task	One call to a module
Play	Maps a group of hosts to tasks
Playbook	YAML file with one or more plays
Role	Reusable, structured bundle of tasks, templates, vars, handlers
Handler	Task triggered by notify only when something changed (e.g. restart service)
Facts	Auto-discovered host data (ansible_os_family , ansible_default_ipv4)
Collection	Distribution format for roles, modules, plugins (amazon.aws , kubernetes.core)

8.2 Setup and ad-hoc commands

```
pip install ansible          # or: sudo apt install ansible
ansible --version
```

Inventory **inventory/hosts.ini**:

```
[web]
web1 ansible_host=10.0.1.11
web2 ansible_host=10.0.1.12

[db]
db1 ansible_host=10.0.2.21 ansible_user=ubuntu

[prod:children]
web
db

[all:vars]
ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/prod.pem
```

Ad-hoc commands (quick one-liners):

```
ansible all -i inventory/hosts.ini -m ping
ansible web -m apt -a "name=nginx state=present update_cache=yes" --become
ansible db -m shell -a "df -h"
ansible web -m service -a "name=nginx state=restarted" -b
ansible all -m setup -a "filter=ansible_distribution*"          # show facts
ansible-inventory -i inventory/hosts.ini --graph
```

ansible.cfg in the project directory sets defaults:

```
[defaults]
inventory = inventory/hosts.ini
host_key_checking = False
forks = 20
[privilege_escalation]
become = True
```

8.3 Playbooks

```
# site.yml
---
- name: Configure web servers
  hosts: web
  become: true
  vars:
    app_port: 8080
    packages: [nginx, git, curl]
  tasks:
    - name: Install packages
      ansible.builtin.apt:
        name: "{{ packages }}"
        state: present
        update_cache: true
        when: ansible_os_family == "Debian"

    - name: Create app user
      ansible.builtin.user: { name: deploy, shell: /bin/bash, groups: sudo, append: true }

    - name: Deploy nginx config from template
      ansible.builtin.template:
        src: nginx.conf.j2
        dest: /etc/nginx/conf.d/app.conf
        owner: root
        mode: "0644"
        validate: "nginx -t -c /etc/nginx/nginx.conf"
      notify: Reload nginx

    - name: Ensure nginx is running and enabled
      ansible.builtin.service: { name: nginx, state: started, enabled: true }

    - name: Wait for the app port
      ansible.builtin.wait_for: { port: 80, timeout: 30 }

  handlers:
    - name: Reload nginx
      ansible.builtin.service: { name: nginx, state: reloaded }
```

Run it:

```
ansible-playbook site.yml # run
ansible-playbook site.yml --check --diff # dry run showing what would change
ansible-playbook site.yml -l web1 -t config # limit to host, run only tagged tasks
ansible-playbook site.yml -e "app_port=9090" -v # extra vars, verbose (-vvv more)
ansible-playbook site.yml --syntax-check
ansible-lint site.yml # best-practice linting
```

Jinja2 template `templates/nginx.conf.j2`:

```
server {
    listen 80;
    server_name {{ inventory_hostname }};
    location / { proxy_pass http://127.0.0.1:{{ app_port }}; }
{% if enable_ssl | default(false) %}
    listen 443 ssl;
{% endif %}
}
```

8.4 Control flow, loops and variables

```
- name: Create several users
  ansible.builtin.user: { name: "{{ item.name }}", groups: "{{ item.groups }}" }
  loop:
    - { name: alice, groups: dev }
    - { name: bob, groups: ops }

- name: Run a command and capture the result
  ansible.builtin.command: cat /etc/os-release
  register: osinfo
  changed_when: false           # read-only command should not report "changed"

- ansible.builtin.debug: { msg: "Running {{ osinfo.stdout_lines[0] }}" }

- name: Only on production
  ansible.builtin.copy: { src: prod.env, dest: /opt/app/.env, mode: "0600" }
  when: env == "prod" and inventory_hostname in groups['web']

- name: Retry until healthy
  ansible.builtin.uri: { url: "http://localhost:8080/health", status_code: 200 }
  register: result
  until: result.status == 200
  retries: 10
  delay: 5

- block:
  - ansible.builtin.command: /opt/app/migrate.sh
  rescue:
  - ansible.builtin.debug: { msg: "Migration failed, rolling back" }
  always:
  - ansible.builtin.debug: { msg: "Done" }
```

Variable precedence (low to high, simplified): role defaults < inventory vars < group_vars < host_vars < play vars < vars_files < role vars < task vars < **--extra-vars** (always wins). Put environment data in **group_vars/prod.yml** and **host_vars/web1.yml**.

8.5 Roles

Roles give structure and reuse. Create with **ansible-galaxy init roles/nginx**:

```
roles/nginx/
  tasks/main.yml      # the work
  handlers/main.yml   # restart/reload handlers
  templates/          # .j2 files
  files/              # static files
  defaults/main.yml   # lowest precedence variables (easy to override)
  vars/main.yml        # higher precedence variables
  meta/main.yml        # dependencies and metadata
```

Use in a playbook, and install community roles/collections from Galaxy:

```
- hosts: web
  become: true
  roles:
    - common
    - { role: nginx, app_port: 8080 }
```

```
ansible-galaxy collection install amazon.aws kubernetes.core
ansible-galaxy install geerlingguy.docker
```

8.6 Secrets with Ansible Vault and dynamic inventory

```
ansible-vault create group_vars/prod/vault.yml      # encrypted file
ansible-vault edit group_vars/prod/vault.yml
ansible-vault encrypt_string 'S3cr3t!' --name 'db_password' # inline encrypted value
ansible-playbook site.yml --ask-vault-pass          # or --vault-password-file ~/.vault_pass
```

Dynamic inventory discovers hosts from the cloud instead of a static file, for example the `amazon.aws.aws_ec2` plugin:

```
# inventory/aws_ec2.yml
plugin: amazon.aws.aws_ec2
regions: [ap-south-1]
filters: { tag:Environment: prod, instance-state-name: running }
keyed_groups: [{ key: tags.Role, prefix: role }]
```

8.7 Example: install Docker and run a container

```
- hosts: app
  become: true
  tasks:
    - name: Install Docker prerequisites
      ansible.builtin.apt: { name: [ca-certificates, curl, docker.io, python3-docker], state: present, update_cache: true }
    - name: Add user to docker group
      ansible.builtin.user: { name: ubuntu, groups: docker, append: true }
    - name: Start Docker
      ansible.builtin.service: { name: docker, state: started, enabled: true }
    - name: Run app container
      community.docker.docker_container:
        name: web
        image: myrepo/web:1.4.2
        state: started
        restart_policy: unless-stopped
        published_ports: ["80:8080"]
        env: { DB_HOST: "{{ db_host }}" }
```

8.8 Performance and best practices

- Keep playbooks **idempotent**: prefer modules (`apt`, `lineinfile`, `template`) over `shell/command`; if you must use them, add `creates:`, `when:` or `changed_when`.
- Use `strategy: free`, increase `forks`, enable SSH **pipelining** and `gather_facts: false` when facts are not needed.
- **Rolling updates**: `serial: 2` (or "25%") updates a few hosts at a time; `max_fail_percentage` aborts on failures.
- Use tags, `--check`, `ansible-lint`, and test roles with **Molecule**.
- **AWX / Ansible Automation Platform** adds web UI, RBAC, scheduling and audit.

Ansible vs Terraform vs Chef/Puppet

	Ansible	Terraform	Chef / Puppet
Main job	Configure servers, deploy apps, orchestrate	Provision infrastructure	Configure servers
Style	Mostly procedural tasks in order, with idempotent modules	Declarative with state	Declarative (Puppet) / Ruby DSL (Chef)
Agent	None (SSH)	None (API calls)	Agent required
State	Stateless	State file	Server-held catalogs
Learning curve	Easy	Moderate	Steeper

Typical combination: **Terraform creates** the VMs, networks and clusters; **Ansible configures** the software inside them (or Terraform outputs feed Ansible's inventory). With container platforms much configuration shifts into images and Kubernetes manifests, but Ansible remains excellent for VMs, bare metal, network devices and glue automation.

Interview questions

- What does idempotent mean? Re-running yields no further change if the system is already in the desired state.
- Handler vs task? A handler runs only if notified by a changed task, once at the end of the play.
- `command` vs `shell`? `shell` supports pipes/redirects through a shell; `command` does not and is safer.
- How do you encrypt passwords? Ansible Vault.
- How does Ansible connect? SSH (Linux) or WinRM (Windows); no agents.

9. Jenkins and CI/CD

9.1 CI/CD concepts

- **Continuous Integration (CI):** developers merge to a shared branch frequently; each change is automatically built and tested, so integration bugs surface within minutes.
- **Continuous Delivery:** every passing build produces a releasable artifact, deployed to staging automatically; production release needs a manual approval.
- **Continuous Deployment:** production release is also automatic when all checks pass.
- **Pipeline:** ordered stages such as Checkout -> Build -> Unit test -> Static analysis -> Package -> Image build/scan -> Push -> Deploy dev -> Integration tests -> Approval -> Deploy prod.
- **Artifact:** versioned output of a build (jar, npm package, Docker image) stored in a repository (Nexus, Artifactory, ECR).
- **Quality gates:** SonarQube, test coverage thresholds and vulnerability scans that fail the build when standards are not met.

9.2 Jenkins architecture

Jenkins is an open-source automation server written in Java with a huge plugin ecosystem.

Component	Role
Controller (master)	Hosts UI, schedules jobs, stores config. Should run few or no builds itself
Agent (node)	Executes build steps; connected via SSH, JNLP/WebSocket, Docker or Kubernetes pods
Executor	A slot on an agent running one build at a time
Job / Project	A configured automation (Freestyle, Pipeline, Multibranch, Folder)
Plugin	Extension: Git, Pipeline, Docker, Kubernetes, Credentials, Blue Ocean
Workspace	Directory where a build runs
JENKINS_HOME	Config, jobs and plugin data (back it up); default <code>/var/lib/jenkins</code>

Installing (Ubuntu or Docker)

```
# Docker (quick start); needs Java 17+ if installing directly on a host
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts-jdk17
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword # first login password
# Package install on Ubuntu (after adding the Jenkins apt repo + key)
sudo apt install -y fontconfig openjdk-17-jre jenkins
sudo systemctl enable --now jenkins # UI on port 8080
```

After the setup wizard: install suggested plugins, create an admin user, configure the URL, then add credentials and agents.

9.3 Declarative Pipeline (Jenkinsfile)

Pipelines as code live in a **Jenkinsfile** in the repository root, so build logic is versioned and reviewed.

```

pipeline {
  agent any                                // or: agent { label 'linux-docker' }
  options {
    timeout(time: 30, unit: 'MINUTES')
    buildDiscarder(logRotator(numToKeepStr: '20'))
    disableConcurrentBuilds()
    timestamps()
  }
  environment {
    IMAGE = "myrepo/web"
    TAG   = "${env.BUILD_NUMBER}-${env.GIT_COMMIT.take(7)}"
    REGISTRY_CREDS = credentials('dockerhub-creds') // username/password credential
  }
  parameters {
    choice(name: 'DEPLOY_ENV', choices: ['dev', 'staging', 'prod'], description: 'Target')
    booleanParam(name: 'SKIP_TESTS', defaultValue: false)
  }
  triggers { pollSCM('H/5 * * * *') } // webhooks are better than polling

  stages {
    stage('Checkout') { steps { checkout scm } }

    stage('Build and Test') {
      parallel {
        stage('Unit tests') {
          when { expression { !params.SKIP_TESTS } }
          steps { sh 'npm ci && npm test' }
          post { always { junit 'reports/**/*.xml' } }
        }
        stage('Lint') { steps { sh 'npm run lint' } }
      }
    }

    stage('SonarQube') {
      steps {
        withSonarQubeEnv('sonar') { sh 'sonar-scanner' }
        timeout(time: 5, unit: 'MINUTES') { waitForQualityGate abortPipeline: true }
      }
    }

    stage('Docker build and push') {
      steps {
        sh '''
          docker build -t $IMAGE:$TAG .
          echo "$REGISTRY_CREDS_PSW" | docker login -u "$REGISTRY_CREDS_USR" --password-stdin
          docker push $IMAGE:$TAG
        '''
      }
    }
  }
}

```

```

    ...
  }
}

stage('Scan image') { steps { sh 'trivy image --exit-code 1 --severity HIGH,CRITICAL $IMAGE:$TAG' } \
}

stage('Deploy') {
  when { branch 'main' }
  steps {
    input message: "Deploy ${TAG} to ${params.DEPLOY_ENV}?", ok: 'Deploy'
    withCredentials([file(credentialsId: 'kubeconfig', variable: 'KUBECONFIG')]) {
      sh 'kubectl set image deployment/web web=$IMAGE:$TAG -n ${DEPLOY_ENV}'
      sh 'kubectl rollout status deployment/web -n ${DEPLOY_ENV} --timeout=120s'
    }
  }
}
}

post {
  success { echo 'Pipeline succeeded' }
  failure { mail to: 'team@company.com', subject: "FAILED: ${env.JOB_NAME} #${env.BUILD_NUMBER}", body \
: "${env.BUILD_URL}" }
  always { cleanWs() }
}
}

```

Declarative vs Scripted pipeline

	Declarative	Scripted
Syntax	Structured: <code>pipeline { stages { stage { steps } } }</code>	Groovy: <code>node { stage('x') { ... } }</code>
Learning curve	Easier, validated up front	Flexible, full Groovy power
Error handling	<code>post { }</code> blocks	<code>try/catch/finally</code>
Recommended	Yes for most cases	Complex dynamic logic only

Useful building blocks

- **Agents:** `agent { docker { image 'node:20-alpine' } }` runs stages inside a container for clean, reproducible builds. `agent { kubernetes { yamlFile 'pod.yaml' } }` spins up a pod per build (Kubernetes plugin).
- **Credentials:** stored encrypted in Jenkins (**Manage Jenkins** -> **Credentials**): username/password, secret text, SSH key, file, certificates. Always use `credentials()` or `withCredentials`; Jenkins masks secrets in logs.
- **Multibranch pipelines** scan a repository, create a job for every branch and pull request containing a Jenkinsfile.
- **Shared libraries** keep reusable Groovy functions in a separate repo: `@Library('my-shared-lib')` _ then `deployApp(env: 'prod')`.
- **Stash/unstash and archiveArtifacts** pass files between stages/agents or keep build outputs.
- **when conditions:** `branch`, `tag`, `changeRequest`, `environment`, `expression`, `anyOf/allOf`.
- `retry(3)`, `timeout`, `catchError` for resilience.

9.4 Triggers and integrations

Trigger	How
Webhook (best)	GitHub/GitLab sends HTTP POST to <code>https://jenkins/github-webhook/</code> on push or PR
Poll SCM	Jenkins checks the repo on a cron schedule (<code>H/5 * * * *</code>)
Scheduled	<code>cron('H 2 * * *')</code> nightly builds
Upstream	Run after another job finishes
Manual / API	Build with parameters, <code>curl -X POST -u user:token https://jenkins/job/x/build</code>

The **H** in cron spreads load by hashing the job name.

9.5 Scaling, security and operations

- **Use agents:** run builds on ephemeral agents (Docker, Kubernetes, EC2 spot fleet) so the controller stays healthy and builds are isolated and clean.
- **Security:** enable authentication (LDAP/SSO/OIDC), role-based authorization (Role Strategy plugin), CSRF protection, restrict script approvals, keep Jenkins and plugins updated, never put secrets in the Jenkinsfile, run behind HTTPS reverse proxy.
- **Configuration as Code (JCasC)** plugin defines Jenkins settings in YAML; use it with plugins lists for reproducible controllers.
- **Backups:** snapshot `JENKINS_HOME`, or use ThinBackup plugin. Monitor with the Prometheus plugin.
- **Common issues:** out of disk (log rotation, `cleanws`), stuck builds (timeouts), plugin conflicts (update in staging first), permission denied for Docker (add `jenkins` user to `docker` group), slow polling (use webhooks).

9.6 Jenkins vs other CI/CD tools

	Jenkins	GitHub Actions	GitLab CI	CircleCI / Azure DevOps
Hosting	Self-hosted (you operate it)	SaaS (and self-hosted runners)	SaaS or self-managed	SaaS
Config	Jenkinsfile (Groovy)	YAML workflows in <code>.github/workflows</code>	<code>.gitlab-ci.yml</code>	YAML
Plugins	1800+, flexible but need upkeep	Marketplace of actions	Built-in features	Orbs / extensions
Setup effort	Highest	Lowest	Low	Low
Best for	Complex/legacy pipelines, on-prem, full control	GitHub-based projects	All-in-one DevOps platform	Managed convenience

Equivalent GitHub Actions workflow:

```
# .github/workflows/ci.yml
name: ci
on: { push: { branches: [main] }, pull_request: {} }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci && npm test
      - uses: docker/login-action@v3
        with: { username: "${{ secrets.DOCKER_USER }}", password: "${{ secrets.DOCKER_TOKEN }}" }
      - uses: docker/build-push-action@v6
        with: { push: true, tags: "myrepo/web:${{ github.sha }}" }
```

GitLab CI equivalent: `stages: [build, test, deploy]` with jobs that define `stage`, `image`, `script`, `only/rules`.

Interview questions

- Freestyle vs Pipeline job? Pipeline is code in a Jenkinsfile with stages, versioned and durable; Freestyle is UI-configured.
- How do you secure credentials? Jenkins Credentials store + `withCredentials`, folder-scoped access, external vaults.
- Controller vs agent? Controller orchestrates; agents run the work.
- How do you trigger a build on push? Webhook from Git server to Jenkins.
- How do you handle build failures? `post { failure {} }` notifications, retries, timeouts, quality gates.

10. Related Topics

10.1 GitOps and ArgoCD

GitOps: Git is the single source of truth for the desired state of the cluster; an in-cluster agent (ArgoCD or Flux) continuously pulls from Git and reconciles. Benefits: audit trail via commits, easy rollback by reverting, no cluster credentials stored in CI (pull instead of push), drift is auto-corrected.

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata: { name: web, namespace: argocd }
spec:
  project: default
  source: { repoURL: https://github.com/org/deploy-repo.git, targetRevision: main, path: apps/web/overlays/prod }
  destination: { server: https://kubernetes.default.svc, namespace: prod }
  syncPolicy: { automated: { prune: true, selfHeal: true }, syncOptions: [CreateNamespace=true] }
```

Typical flow: Jenkins/Actions builds and pushes an image, then updates the image tag in the deployment repo; ArgoCD detects the commit and syncs the cluster.

10.2 Monitoring, logging and observability

Pillar	Question	Tools
Metrics	How is it performing? (numbers over time)	Prometheus, Grafana, CloudWatch, Datadog
Logs	What happened?	ELK/EFK (Elasticsearch, Fluentd/Fluent Bit, Kibana), Loki, CloudWatch Logs
Traces	Where did the request spend time across services?	Jaeger, Tempo, OpenTelemetry, X-Ray

Prometheus scrapes `/metrics` endpoints on a pull model, stores time series, and evaluates alert rules (Alertmanager routes to Slack, PagerDuty, email). Query with **PromQL**:

```
rate(http_requests_total{status!="5.."}[5m]) # error rate per second
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le)) # p95 latency
100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100) # CPU %
```

Install on Kubernetes with `helm install kube-prometheus-stack prometheus-community/kube-prometheus-stack`.

- **Golden signals** (Google SRE): latency, traffic, errors, saturation. **SLI** (measurement), **SLO** (target, e.g. 99.9% success), **SLA** (contractual promise), **error budget** (allowed unreliability).
- Alert on symptoms users feel, not every cause; every alert needs an owner and a runbook.

10.3 DevSecOps (security in the pipeline)

Practice	Tools
Secret scanning	gitLeaks, trufflehog, GitHub secret scanning
SAST (static code analysis)	SonarQube, Semgrep, CodeQL
Dependency scanning (SCA)	Dependabot, Snyk, OWASP Dependency-Check
Container image scanning	Trivy, Grype, ECR scanning
IaC scanning	tfsec, Checkov, KICS
DAST (running app tests)	OWASP ZAP
Runtime security	Falco, admission controllers (Kyverno, OPA Gatekeeper)
Secrets management	HashiCorp Vault, AWS Secrets Manager, External Secrets Operator

Practice	Tools
Supply chain	SBOMs (Syft), image signing (cosign), SLSA provenance

10.4 Artifact repositories and registries

Nexus and JFrog Artifactory store jars, npm, pip and Docker artifacts; cloud registries (ECR, ACR, GAR, GHCR) store images. Use immutable tags (git SHA or semantic version), retention policies and vulnerability scanning on push.

10.5 Networking and web server basics for DevOps

- **DNS** maps names to IPs (A, AAAA, CNAME, MX, TXT records); TTL controls caching.
- **HTTP**: methods (GET, POST...), status codes (2xx ok, 3xx redirect, 4xx client error, 5xx server error), headers, TLS handshake and certificates (Let's Encrypt via certbot or cert-manager).
- **Reverse proxy and load balancer**: Nginx example:

```
upstream app { server 10.0.1.11:8080; server 10.0.1.12:8080; }
server {
    listen 80;
    server_name app.example.com;
    location / {
        proxy_pass http://app;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

- **CIDR quick math**: $/32 = 1$ address, $/24 = 256$, $/16 = 65,536$. Private ranges: `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`.
- **Layer 4 vs Layer 7**: TCP/UDP-level balancing vs HTTP-aware routing.

10.6 YAML and JSON essentials

YAML is whitespace sensitive (spaces only, never tabs) and is the language of Kubernetes, Ansible, Compose and CI files.

```
name: demo                # scalar
ports: [80, 443]          # inline list
env:                      # map
  LOG_LEVEL: info
  FEATURES:
    - search              # block list
    - export
script: |                 # multi-line, keeps newlines
  echo one
  echo two
base: &base { retries: 3 } # anchor
job: { <<: *base, timeout: 30 } # merge with alias
```

Validate with `yamllint`, `kubectl apply --dry-run=client`, or `yq`. Quote strings like "yes", "on", "1.0" to avoid unintended type conversion.

10.7 Service mesh and advanced Kubernetes ecosystem (awareness)

Istio/Linkerd add mTLS, traffic shaping, retries and telemetry between services using sidecar proxies. **Operators** extend Kubernetes with custom resources to manage complex software (Postgres, Kafka). **Argo Rollouts/Flagger** automate canary and blue/green. **KEDA** autoscales on events (queue length). **Crossplane** manages cloud infrastructure through Kubernetes APIs.

10.8 Release engineering and reliability practices

- **Semantic versioning** **MAJOR.MINOR.PATCH**; tag releases in Git, build once and promote the *same* artifact through environments.
- **Feature flags** decouple deploy from release; **database migrations** (Flyway, Liquibase) should be backward compatible (expand, migrate, contract).
- **Rollback plan** for every release; practice it.
- **Incident management**: detect, triage, mitigate, communicate, resolve, then a blameless post-mortem with action items.
- **Chaos engineering** (Chaos Monkey, Litmus) deliberately injects failure to prove resilience.
- **DORA metrics** measure delivery performance: deployment frequency, lead time for changes, change failure rate, time to restore service.

11. End-to-End Project: From Commit to Production

11.1 Architecture

A developer pushes code to **GitHub**. A webhook triggers **Jenkins**, which builds and tests the app, builds a **Docker image**, scans it, pushes it to **ECR**, and updates the deployment. The infrastructure (VPC, EKS cluster, ECR, RDS) is created by **Terraform**; the supporting VMs (Jenkins server, bastion) are configured by **Ansible**; the app runs on **Kubernetes** and is observed with **Prometheus and Grafana**.

Step	Tool	Action
1	Terraform	<code>terraform apply</code> creates VPC, subnets, EKS, ECR, IAM roles, RDS
2	Ansible	Installs and configures Jenkins, Docker, kubectl, trivy on the CI VM
3	Git	Developer opens PR; checks run; merge to <code>main</code>
4	Jenkins	Build, test, SonarQube gate, docker build, Trivy scan, push image tagged with commit SHA
5	Kubernetes	<code>kubectl set image</code> or Helm upgrade performs a rolling update; readiness probes gate traffic
6	Monitoring	Prometheus alerts on error rate and latency; Grafana dashboards; logs to Loki

11.2 Repository layout

```
repo/
  app/                # source code + Dockerfile
  Jenkinsfile
  k8s/ or helm/       # manifests or chart
  infra/terraform/
    envs/dev  envs/prod # per-environment root modules
    modules/vpc modules/eks
  ansible/
    inventory/  roles/jenkins  roles/docker  site.yml
```

11.3 Command sequence to practice

```
# 1. Infrastructure
cd infra/terraform/envs/dev && terraform init && terraform plan -out=tf.plan && terraform apply tf.plan
aws eks update-kubeconfig --name dev-cluster --region ap-south-1

# 2. Configure CI server
cd ansible && ansible-playbook -i inventory/aws_ec2.yml site.yml --limit ci

# 3. Build and ship manually once (what Jenkins will automate)
docker build -t $ECR/web:v1 app/ && docker push $ECR/web:v1
helm upgrade --install web helm/web -n dev --create-namespace --set image.tag=v1

# 4. Verify
kubectl rollout status deploy/web -n dev && kubectl get ingress -n dev
curl -I https://dev.example.com/health
```

12. Quick Reference and Study Plan

12.1 One-page command cheat sheet

Tool	Most-used commands
Linux	<code>ls -lah, grep -rn, find, tail -f, ps aux, top, df -h, du -sh, ss -tulpn, chmod/chown, systemctl, journalctl -u, curl -v</code>
Git	<code>status, add -p, commit, switch -c, merge/rebase, log --oneline --graph, stash, reset/revert, reflog, cherry-pick</code>
Docker	<code>build, run -d -p, ps, logs -f, exec -it, compose up -d, system prune, push</code>
Kubernetes	<code>get/describe/logs/exec, apply -f, rollout status/undo, scale, port-forward, top, config use-context</code>
Terraform	<code>init, fmt, validate, plan, apply, destroy, state list/mv/rm, import, workspace</code>
Ansible	<code>ansible -m ping, ansible-playbook --check --diff, -l, -t, ansible-vault, ansible-galaxy</code>
Jenkins	<code>Jenkinsfile</code> stages, <code>credentials()</code> , <code>withCredentials</code> , <code>parallel</code> , <code>post</code> , <code>input</code> , shared libraries
AWS	<code>sts get-caller-identity, ec2 describe-instances, s3 sync, eks update-kubeconfig</code>

12.2 Concept comparison summary

Question	Answer
Docker vs VM	Containers share a kernel (light, fast); VMs run full OSes (strong isolation)
Terraform vs Ansible	Provision infrastructure vs configure what runs on it
Pod vs Deployment	One running unit vs a controller managing replicas and rollouts
Jenkins vs GitHub Actions	Self-hosted flexible engine vs hosted, repo-native workflows
Push vs pull deployment	CI pushes to cluster vs GitOps agent pulls from Git
Mutable vs immutable infra	Patch servers in place vs replace with new images
Blue/green vs canary	Switch all traffic between two environments vs shift a small percentage gradually
Scale up vs out	Bigger machine vs more machines
RPO vs RTO	Acceptable data loss vs acceptable downtime

12.3 A practical 8-week study plan

- Week 1:** Linux commands, permissions, processes, shell scripts, SSH; practice on a free VM or WSL.
- Week 2:** Git workflows, branching, PRs, resolving conflicts; contribute to a small repo with a teammate.
- Week 3:** Cloud basics (AWS free tier): EC2, S3, IAM, VPC, load balancer, billing alarms.
- Week 4:** Docker: write Dockerfiles, multi-stage builds, Compose a 3-tier app.
- Week 5:** Kubernetes: kind/minikube, Deployments, Services, Ingress, ConfigMaps, probes, HPA, Helm.
- Week 6:** Terraform: VPC + EC2 + S3 with remote state, modules, workspaces; then an EKS cluster.
- Week 7:** Ansible: roles, templates, vault, dynamic inventory; configure the Jenkins server with it.
- Week 8:** Jenkins pipeline end to end plus monitoring, then rebuild the whole project from scratch without notes.

Tip: do not just read. Break things on purpose (kill a pod, fill a disk, misconfigure a security group) and practice diagnosing them. Keep every lab in Git, destroy cloud resources afterwards (`terraform destroy`) to avoid bills, and explain each tool aloud as if teaching someone: that is what interviews test.

12.4 Frequently asked scenario questions

- **A deployment is stuck at 0 pods ready. What do you check?** `kubectl describe` for events, image pull errors, probe failures, resource requests vs node capacity, Secrets/ConfigMaps present, then logs.
- **The site is down after a release.** Check monitoring and recent deploys; `kubectl rollout undo` to restore service first, investigate afterwards.
- **How would you design CI/CD for a microservices app?** One pipeline per service from Git, build once, immutable image tags, automated tests and scans, deploy via Helm/GitOps to dev -> staging -> prod with approvals, observability and automated rollback on failed health checks.
- **How do you manage secrets across the toolchain?** Never in Git; vault/secrets manager, short-lived OIDC credentials, Jenkins credentials store, Kubernetes External Secrets, encrypted Ansible Vault and sensitive Terraform variables with protected state.
- **How do you reduce cloud cost?** Tagging and budgets, right-sizing, auto-scaling, Spot for CI/batch, schedules to turn off non-prod, storage lifecycle policies, Savings Plans for steady load.